

Database at a Glance — K Orchids · Project Green

PostgreSQL 18 · a normalised, integrity-first schema where the engine — not the app — guarantees correctness. All figures below are read directly from the live database.

38
TABLES

128
INDEXES

44
CHECK

18
UNIQUE

60
FOREIGN KEYS

31
SEQUENCES

13
TRIGGERS

1 · The 38 tables — by domain

DOMAIN	TABLES
Identity & Access — login, sessions, RBAC	roles · permissions · role_permissions · role_access_windows · users · auth_sessions · login_history · email_tokens
Catalogue & Inventory — what's sold & stock	suppliers · categories · products · product_images · bulk_pricing_tiers · stock_movements · stock_alerts
Pricing — history & governance	price_history · price_change_requests
Trade & Orders — buyer → order	buyer_tiers · trade_accounts · carts · cart_items · rfqs · rfq_items · orders · order_items
Money — billing & payments	invoices · payments · invoice_adjustments
Fulfilment & Returns	rma_requests · rma_items · deliveries · delivery_events
Cross-cutting — platform	audit_logs · notifications_outbox · settings · cms_blocks · job_runs · bloom_events

2 · Functions, triggers & sequences

OBJECT	WHY IT EXISTS
set_updated_at() — trigger function	The one custom function. A BEFORE UPDATE trigger on every mutable table stamps updated_at = NOW() automatically — "last modified" is always correct and never depends on app code remembering. 13 triggers use it (orders, invoices, products, trade_accounts, rfqs, rma_requests, deliveries, carts, suppliers, categories, users, price_change_requests, cms_blocks).
31 sequences	One per table with an integer primary key. They hand out PKs atomically, so concurrent inserts can never collide on an id — race-free identifiers with zero app logic.
Append-only ledgers	audit_logs · stock_movements · price_history are treated as append-only (insert-only) by design; production hardening (role-independent guards) is defined in the reconciliation migration.

No classic stored procedures — on purpose. Multi-step business logic (approve order, record payment, quote→order) runs in the **application service layer** inside explicit transactions with `SELECT ... FOR UPDATE`. This keeps the logic **version-controlled, unit-testable and reviewable**, while the database enforces *integrity* declaratively through constraints, the updated-at trigger and sequences. Procedural DB code is reserved for the trigger that genuinely belongs there.

3 · Constraints — correctness enforced by the engine

CONSTRAINT	COUNT	WHAT IT GUARANTEES
CHECK	44	Only valid values are stored — status enums (e.g. order/invoice/RMA states), positive prices & quantities, and the key invariant <code>reserved_qty <= stock_qty</code> (stock can never be over-reserved).
UNIQUE	18	No duplicates — one invoice per order, unique document numbers, and <code>uq_payments_idempotent</code> (<code>invoice_id</code> , <code>method</code> , <code>reference</code>) so a double-clicked or re-sent payment can't be recorded twice.
FOREIGN KEY	60	Referential integrity — an order line can't point at a missing product; cascades clean up children safely.
PRIMARY KEY	38	Every table is uniquely addressable (UUID or sequence-backed integer).

4 · Indexes & the I/O improvement

128 indexes (8 **partial**, 2 **GIN trigram**) placed on the columns the app filters, joins and sorts on — turning *sequential scans* (read every row, O(N)) into *index lookups* (B-tree height $\approx \log N$).

KEY INDEX	USED FOR
<code>products(name), products(sku)</code> — GIN trigram	Fuzzy catalogue search (<code>ILIKE '%pha1%'</code>).
<code>orders(buyer_id, status)</code>	Buyer dashboards & admin work-queues.
<code>invoices(due_date) WHERE status IN(...)</code> — partial	Overdue / aging cron, indexing only unpaid rows.
<code>auth_sessions(refresh_token_hash)</code>	Validated on every token refresh (hot path).
<code>login_history(email, occurred_at) WHERE success=false</code> — partial	Brute-force detection on recent failures only.
<code>UNIQUE(invoice_id, method, reference)</code>	Fast lookup <i>and</i> payment idempotency.

Measured: from sequential scan → index (EXPLAIN ANALYZE on `orders`).

```
orders WHERE buyer_id = ? AND status = 'APPROVED'
BEFORE (no index) → Seq Scan · reads all 18 data pages · discards 930 of 958 rows (~97% wasted)
AFTER  (idx_orders_buyer_status) → Bitmap Index Scan · ~5 heap blocks · 28 rows
```

Why it matters at scale. The sequential-scan cost is **linear** in table size. At **1,000,000 orders** (~150 MB $\approx$ 18,500 pages of 8 KB), an unindexed dashboard query would read **all ~18,500 pages (~150 MB)** every time; the composite B-tree (height $\approx$ 3) answers in **~5 page reads (~40 KB)** — roughly a **3–4 orders-of-magnitude** reduction in I/O that stays flat as the data grows. Partial indexes shrink this further by indexing only the rows a query ever needs (e.g. unpaid invoices, failed logins).

~97%

ROWS A SEQ SCAN WASTES

18→5

PAGES READ (DEMO)

~3–4×10ⁿ

LESS I/O AT 1M ROWS

8 / 2

PARTIAL / GIN INDEXES